

WZPlanner: Safe End-to-End Path Planning for Autonomous Driving in Work Zones

Nishad Sahu Changzhong Qian Guangzhou Cai
Shounak Sural Rangunathan (Raj) Rajkumar
Department of Electrical and Computer Engineering
Carnegie Mellon University, Pittsburgh, PA, USA

Abstract

Work zones are common safety-critical driving environments where lane geometry is frequently altered by temporary traffic control artifacts. Partial or complete lane or road closures in work zones may not be reflected in on-board maps creating both perception and planning challenges for autonomous vehicles (AVs). Existing perception systems and modular AV stacks often fail to generalize well in such settings, due to structural complexities of possible road configurations and in part due to the limited availability of publicly released data with structured geometric supervision. This paper presents the **WorkZonePlan** dataset, a work zone dataset comprising **149K+ synthetic** and **5K+ real-world** multimodal samples with **3D annotations** for lane boundaries, work zone boundaries, and driving trajectory planning options. The dataset also provides 76 closed-loop work-zone scenarios in CARLA, each replayed under three weather conditions to form 228 Bench2Drive-format evaluation routes. We also introduce **WAVE** (Work-zone-focused AV data generation in Virtual and rEal Environments), a semi-automated pipeline used to create the **WorkZonePlan** dataset. Finally, we propose **BoundaryFormer (BF)**, a transformer-based architecture that jointly predicts lane and work zone boundary polynomials and driving trajectories.

BF uses slot attention for boundary prediction. Ablation studies show that a distinct trajectory decoder leveraging boundary slot features improves trajectory prediction substantially compared to predicting trajectories purely within the slot-attention framework. Building on this finding, we introduce **BF++**, a more robust successor with Camera and Camera+LiDAR variants. **BF++** adds work-zone-specific geometric structure through metric ground-plane encoding, typed boundary/trajectory queries, long-range point anchors, image-space curve refinement, and conservative gated LiDAR fusion. Over the 211 routes common to all four compared models at the evaluation freeze,



Figure 1. Visualization of camera annotation on some real-world data-points in **WorkZonePlan** dataset along with the bird's-eye view of the annotations on the right. (Grey = Driving trajectory, Green = Lane boundaries, Red/Orange = Work zone boundaries)

BF++-Camera and **BF++-Camera+LiDAR** obtain Driving Scores of 63.0 and 64.4, respectively, compared with 59.3 for **SimLingo** and 26.1 for **TransFuser++ (TF++)**. **BF++** is 40× smaller than **SimLingo** and more than 10× smaller than **TF++**, while achieving higher Driving Scores. Our results show that a unified end-to-end model prediction of lane, work zone boundaries, and planning trajectories is a promising direction toward safer AV operation in work zones. Code and dataset are available at <https://github.com/Nishad-Sahu/WZPlanner>.

1. INTRODUCTION

Autonomous driving technology has made significant strides on structured highways and urban roads, where lane geometry, traffic control and maps are relatively stable. Work zones, however, violate many of these assumptions simultaneously: lanes may narrow or shift, shoulders may temporarily become travel lanes, and detours that are not reflected on maps may appear. Entry and exit ramps may be shifted, and/or intersect a long work zone. In addition, workers and construction vehicles often occupy atypical positions within or near the travel way. These challenges are frequently compounded by adverse weather and reduced



Figure 2. Some synthetic data-points with 3D polynomial annotations in *WorkZonePlan* dataset across different work zone scenarios with different work zone objects in various visibility and weather conditions. (Driving trajectory - Red, work zone boundary - blue, lane boundaries - yellow)

visibility. As a result, safe navigation in dynamic and unpredictable environments such as work zones remains an open problem for autonomous vehicles (AVs).

According to the Federal Highway Administration (FHWA), work zones contribute substantially to traffic congestion and safety risk, motivating specialized solutions to ensure reliable AV operation [1]. FHWA statistics indicate that, in the United States, work zones are associated with approximately one fatality per 4 billion vehicle-miles traveled and per \$112 million in roadway construction expenditures. In 2022, there were 891 work zone-related fatalities, including 742 drivers and passengers and 145 bicyclists and pedestrians [1]. Therefore, reliable and validated techniques are needed to detect and navigate a wide range of complex work zone scenarios. Meeting this challenge also needs an exhaustive suite of scenarios for evaluating AV behavior both in simulation, closed test-tracks and in real-world deployments.

1.1. From Rule-Based Decisions to End-to-End Prediction

Most prior studies on safe AV navigation in work zones adopt a modular architecture [13–16], with distinct modules for work zone perception (e.g., object and boundary detection), lane detection, behavior planning, path planning, and other downstream decision-making tasks. For example, [14] studies a broad range of work zone scenarios defined by a state transportation authority and summarizes behavioral, perception, path-planning, and route-planning requirements in a consolidated, rule-based framework. Their system explicitly switches among *map following*, *lane following*, and *work zone boundary following* depending on the observed context. While such rules-driven designs offer transparency, explainability, explicit controllability and align with operational safety requirements, they do have a key limitation: robust performance depends on correctly

identifying the current mode and on engineered switching logic, which may not generalize to the long tail of work zone configurations. Moreover, mathematical or heuristic boundary construction methods such as convex-hull grouping in [15] can incorrectly merge work zone objects on opposite sides of the roadway into a single boundary, yielding false positives for road blockage even though these work zone objects belong to two distinct boundaries and represent a drivable corridor.

Destination-level navigation and local trajectory prediction are complementary. Airinei et al. use graph-based path generation, explainable augmentation, and curriculum learning for target-conditioned visual navigation [2], while Leordeanu and Paraicu combine visual localization with destination-conditioned trajectory prediction [9]. **BoundaryFormer** instead predicts local work-zone boundaries and trajectory options which can be later combined with a higher planning block for navigation to destination. In our CARLA closed-loop evaluation, every baseline model and **BoundaryFormer** receives the same L/R/straight navigation command to select a local trajectory planning options from multiple options based on the goal location.

The relative strengths and weaknesses of a layered architecture *vs* a consolidated approach motivate a complementary question explored in this paper: *Can a single neural network directly predict feasible driving trajectory options from perception inputs in work zones?* If so, a conventional route planner (which we treat as standard) can select among the network’s predicted trajectory candidates based on route-level objectives, while the perception-to-trajectory network handles local geometry and safety constraints. This paper makes the following contributions:

- **WAVE** (Work-zone-focused AV data generation in Virtual and rEal Environments): a scalable, semi-automated pipeline that generates work-zone scenarios and produces **3D annotations** for lane boundaries, work-

Table 1. Comparison of work-zone datasets and benchmarks. *WorkZonePlan* has 76 closed-loop CARLA scenarios, replayed in three weather conditions to form 228 evaluation routes, together with calibrated 3D lane/work-zone boundaries and trajectory options across adverse conditions in synthetic and real-world environments.

Dataset	Venue (Year)	Cam	LiDAR	Real Synth	Calib. 3D GT	Lane Geom. GT	WZ Contours GT	Traj./ Path GT	Adverse Weather/ Visibility	Closed Loop WZ Scen.	#WZ Obj.	Dataset Size and description
nuScenes [4]	CVPR '20	✓	✓	✓	✗	✓	✗	✗	✗	✗	3	40k samples; work-zone object classes are sparse
BDD100K [19]	CVPR '20	✓	✗	✓	✗	✗	✗	✗	✓	✗	0	100k samples; no work-zone object classes
Shi et al. [15]	ITSC '21	✓	✓	✓	✗	Partial	✗	✗	✗	✗	4	757 nuScenes samples with WZ labels
RZDG [18]	IV '25	✓	✓	✓	✓	✓	✗	✓	✗	✗	2	8,591 synthetic + 1,357 real frames
ROADWork [5]	ICCV '25	✓	✗	✓	✗	Partial (recon.)	✗	Partial (recon.)	✓	✗	15	4,375 videos; 9,650 keyframes; 129k path images; 5,430 with reconstructed 3D trajectories
WorkZone3D [17]	WACV '26	✓	✓	✓	✗	✓	✗	✗	✓	✗	3	21,650 real samples
WorkZonePlan (Ours)		✓	✓	✓	✓	✓	✓	✓	✓	✓	9	149,478 synthetic + 5,178 WorkZone3D-derived real samples (154,656 total)

Note: "Designed WZ Scen." counts purpose-built layouts, not frames, routes, or observed sites (e.g., ROADWork's > 5K work zones); N/R = no comparable count reported. "Partial (recon.)" = reconstructed rather than direct calibrated 3D labels. WZ = work zone; GT = ground truth; Calib. = calibrated; Geom. = geometry; Traj. = trajectory; Obj. = objects.

zone boundaries, and driving trajectories in simulation and on real data.

- **WorkZonePlan dataset:** We introduce **149,478 synthetic** and **5,178 real** samples, for **154,656 samples** in total (Figs. 2 and 1); the latter are an exact open-source WorkZone3D subset [17] with added boundary and trajectory labels. The closed-loop benchmark contains 76 work-zone scenarios distributed across CARLA Towns01–05 and Town10 and replayed under three weather conditions, yielding 228 evaluation routes. The dataset covers work zone scenarios designed manually using transportation-authority-inspired layouts spanning lane changes, merges, exits, detours, and forks, nine work-zone object types, and nine weather/visibility combinations (Day/Dusk/Night $\times$ No/Medium/Heavy rain).
- **BoundaryFormer (BF):** We propose a transformer-based architecture that jointly predicts lane boundaries, work-zone boundaries, and driving trajectory options from perception inputs, using slot attention for structured scene parsing and a dedicated trajectory decoder for global, long-horizon reasoning. This can be used as a baseline model for the *WorkZonePlan* dataset by researchers.
- **BoundaryFormer++ (BF++):** Building on the superior *BF-DistinctPlanner* result, we develop camera-only and camera+LiDAR model with metric, type-aware, and topology-oriented inductive biases for work-zones.
- **Experimental evaluation and ablation study:** We evaluate BF++ in open loop and on a closed-loop benchmark of 228 routes spanning six towns and three weather conditions, with external driving baselines and separate camera versus camera+LiDAR results. The four-model comparison uses the 211 routes common to all methods at the evaluation freeze; complete 228-route results are reported in the supplementary material.

1.2. Problem Statement

We consider the task of jointly predicting (i) lane boundaries, (ii) work zone boundaries, and (iii) driving trajectory

options through the work zone directly from perception. BF uses a monocular RGB image, whereas BF++ is evaluated in both RGB-only and RGB+LiDAR forms. The outputs are structured geometric entities represented as parametric polynomials (third-order polynomial coefficients with valid spatial ranges).

2. Related Work

2.1. Work-zone Datasets and Benchmarks

Work zones have recently gained attention as a common yet under-explored operational domain in autonomous driving. Common AV datasets such as nuScenes [4] and BDD100K [19] include work-zone-related object classes (e.g., cones and barriers), but these scenarios are relatively sparse and typically lack structured supervision for work-zone geometry [15]. The ROADWork dataset [5] provides a relatively large-scale real-world benchmark for recognizing and analyzing work zones, including annotated images and videos from thousands of work zones across multiple cities, and tasks such as object/sign detection and pathway estimation. ROADWork notes that foundation models and standard perception pipelines may fail in these scenarios, underscoring the need for dedicated datasets and models. However, ROADWork primarily provides 2D annotations for work-zone objects/signs and driving pathways; its 3D pathway labels are derived via 3D reconstruction rather than directly captured using calibrated LiDAR and GPS positioning sensors. The RZDG dataset [18] provides both real and simulated work zone data but is only limited to 2 work zone object classes. All these datasets lack varying weather and visibility conditions and are mostly collected under good operating conditions. The WorkZone3D dataset [17] provides camera+Lidar work zone data in bad weather conditions like day, night, rain, snow but lacks trajectory annotations for path planning and has only three work zone object classes.

WorkZonePlan, the dataset presented in this paper is complementary to the existing datasets. Table 1 presents

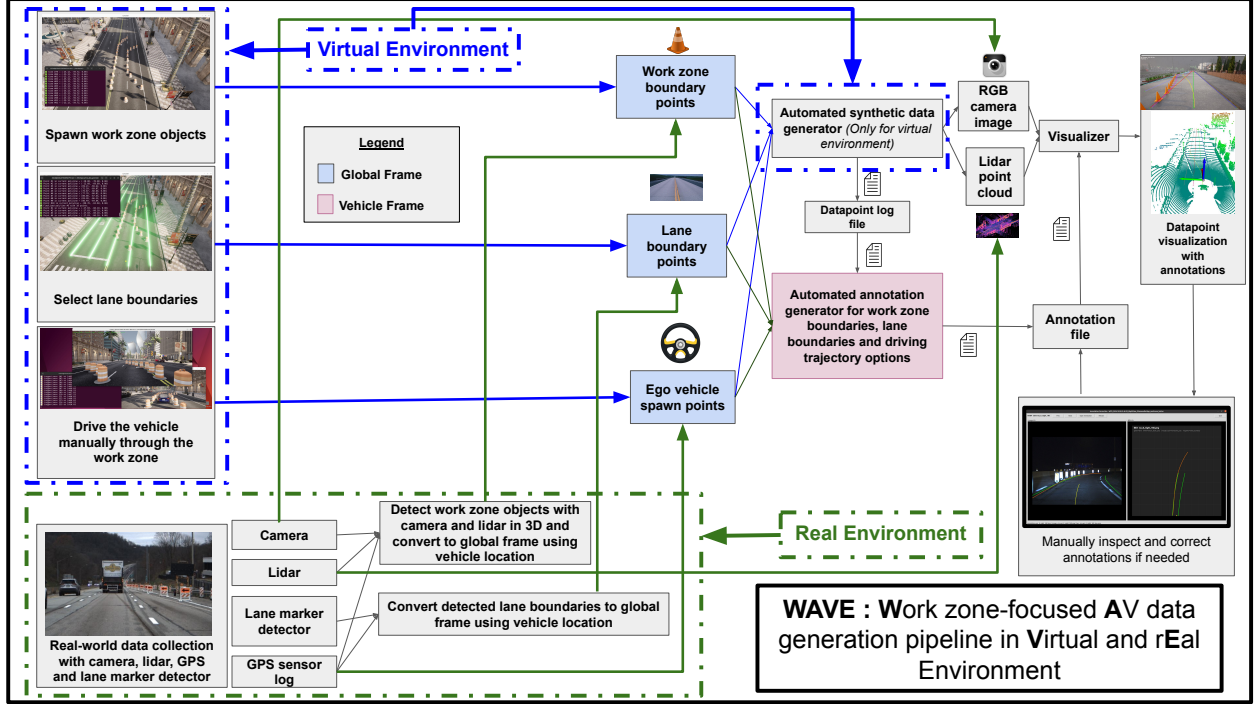


Figure 3. Overview of **WAVE**: Work zone-focused AV data generation in Virtual and rEal Environments

a comparison of our dataset with other state-of-the-art autonomous driving and work zone datasets. It has a variety of adverse weather and visibility conditions across different work zone scenarios. It also provides annotations for lane geometry and work-zone boundary contours, both which are critical for predicting feasible driving trajectories in reconfigured road layouts. In other words, our focus is on *fused (or one-shot) geometric supervision* for *boundaries and trajectories*. The dataset has **3D annotations** and calibrated sensor metadata, enabling precise evaluation of boundary localization and trajectory feasibility under controlled scenario attributes. The dataset also combines **large-scale synthetic data** generation for broad coverage supplemented by **real-world AV data** to support domain adaptation and transfer experiments. Finally, its 76 Bench2Drive-style [8] work-zone scenarios are based on transportation-authority specifications and are replayed in three weather conditions to form 228 closed-loop routes, capturing complex, operationally realistic work-zone configurations.

The dataset differs from standard lane datasets by explicitly representing *work-zone boundaries* as distinct entities, enabling models to reason about temporary traffic-control geometry that may not align with painted lane markings.

2.2. Boundary Detection and Trajectory Prediction

Lane marker detection has evolved from hand-crafted feature pipelines to deep learning approaches that regress polynomials or splines in the image plane and, more recently, to estimating 3D lane geometry [3]. Recent state-of-the-

art methods increasingly also leverage transformers for 3D lane prediction [10, 11]. While several studies detect work-zone objects [5, 15, 16], comparatively few address the prediction of *work-zone boundary contours*. For instance, [15] proposes inferring work-zone contours from detected work-zone objects.

Trajectory prediction is widely studied in autonomous driving [6], where multiple future trajectories are plausible due to route choice, interactions, and topology. However, many trajectory predictors assume stable lane geometry and/or rely on HD maps. Work zones however violate these assumptions by introducing additional constraints and an altered set of feasible paths. The need to detect such changes motivates *joint boundary-and-driving-trajectory prediction*, where a model can infer both the drivable corridor boundaries and candidate trajectories directly from perception. To the best of our knowledge, no prior work jointly addresses lane-boundary detection, work-zone boundary detection, and multi-modal trajectory prediction. We utilize a unified transformer-based architecture with attention to accomplish this objective.

3. WAVE: Dataset Generation Pipeline

We present **WAVE** (Work-zone-focused AV data generation in Virtual and rEal Environments), a semi-automated pipeline that enables scalable generation of work zone driving data on public roadways as well as in a virtual environment like CARLA. It generates calibration-consistent 3D geometric annotations for work zone objects, work zone

boundaries, lane boundaries and possible driving trajectory options (see Figs. 2,4).

The overview of the **WAVE** data generation pipeline is presented in Fig. 3. **WAVE** combines lightweight human-in-the-loop specification with automated data capture and labeling. In the synthetic world an operator first uses a GUI in the CARLA spectator view to (i) spawn work zone assets, and (ii) select relevant lane boundaries. Then, the operator manually drives the ego vehicle through the work zone to record a representative path. These interactions provide three inputs in the global frame: work zone boundary points, lane boundary points, and ego trajectory waypoints.

Given these inputs, **WAVE** executes an automated synthetic data generator that replays the scene and captures synchronized RGB images and LiDAR point clouds under diverse environment settings: time (day, dusk, night), weather (rain/no rain) and variety of work zone objects (cones, barriers, barrels, fences etc.). All sensor data are logged alongside the required calibration metadata. Next, an automated annotation module transforms global-frame boundary points into an ego-centric coordinate frame and compares the actual 3D depth of the boundary points with the depth of the corresponding pixels in the camera image. If both distances are not within a certain threshold (say 25 cm) then it is highly likely that the annotation point is occluded from the RGB camera perspective and hence is dropped. Then, based on the filtered data, the script generates 3D annotations for lane boundaries, work zone boundaries, and driving trajectory options. Finally, **WAVE** produces a compact annotation file per datapoint and a visualization overlay (as shown in Fig. 4) for rapid quality assurance, enabling efficient iteration over scenario variations and large-scale dataset creation. For real data, **WAVE** uses a 5,178-frame WorkZone3D subset [17], retaining its synchronized sensors, GPS-RTK/vehicle state, calibration, and 3D boxes. It converts inherited boxes and lane detections to a common global frame to add work-zone/lane boundaries and trajectories, with GUI-based inspection and correction.

4. The WorkZonePlan Dataset

Scenario Families and Transportation Authority-Specified Designs: **WorkZonePlan** dataset has manually designed work-zone scenario layouts organized into families based on transportation-authority specifications. These include single/multi-lane changes, merges, exits, detours and forks. The members of each scenario family are grouped together because they induce unique changes in feasible driving corridors and a similar behavior maybe required from the AV.

Work Zone Object Library: We use a curated set of **nine** work zone/boundary objects including cones, barrels, horizontal panels, fences, zipper barriers, vertical flat panels and multiple concrete barrier variants (Supplementary

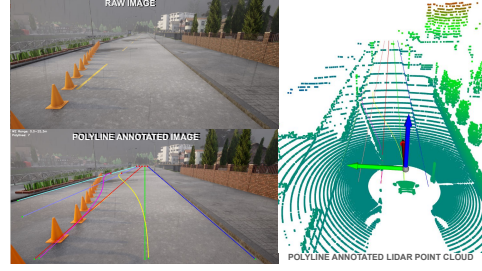


Figure 4. Visualization of Camera and Lidar annotation on a synthetic datapoint in **WorkZonePlan** dataset.

Fig. S2). These assets are procedurally placed to form work zone boundaries consistent with the intended scenario topology.

Weather and Visibility Conditions: To evaluate robustness under adverse perception conditions, **WorkZonePlan** includes **nine** weather/visibility combinations formed by Day/Dusk/Night lighting crossed with No/Medium/Heavy rain. The real-world data has day/night and rain/no-rain.

Sensors and annotations: Each frame includes synchronized camera/LiDAR data and **3D annotations** represented as third-order polynomials with valid ranges and objectness. The real subset retains WorkZone3D’s six 16-channel VLP-16 LiDARs and front RGB camera: 1024×768 ($25.3^\circ \times 17.3^\circ$). Camera/LiDAR acquisition is 30/20 Hz or 10/10 Hz. We use precise timestamps and 6-DoF UTM state support alignment, with OpenCV, cross-path, and object-guided calibration as discussed in [17]. We add lane and work-zone boundaries along with driving trajectories for training and evaluation.

Dataset composition: **WorkZonePlan** has **137,749-frame** Town01/Town10 synthetic corpus and adds a leakage-resistant **11,729-frame** held-out set from Town02,03,04,05 (2,797/4,945/1,792/2,195 frames, respectively). The resulting release contains **149,478 synthetic** CARLA frames and **5,178 real-world** frames. The original corpus contains 80K+ Town01 and 50K+ Town10 frames and provides the in-distribution data, while Town02–05 are reserved for out-of-distribution evaluation. The closed-loop benchmark uses 12 base scenarios in each of Towns01–05 and 16 additional in-distribution scenarios in Town10. Replaying every scenario under clear, fog, and storm conditions gives 76 base scenarios and 228 evaluation routes. Supplementary Sec. S6 and Table S4 provide the town-wise frame, modality, split, and closed-loop distribution; Supplementary Sec. S5 distinguishes the 211-route common comparison from the complete 228-route records. The real samples are derived from WorkZone3D; all source modalities and 3D boxes are retained, and our structured boundary and driving trajectory labels are added. We use synthetic data for training and ablations and real data for fine-tuning and domain transfer.

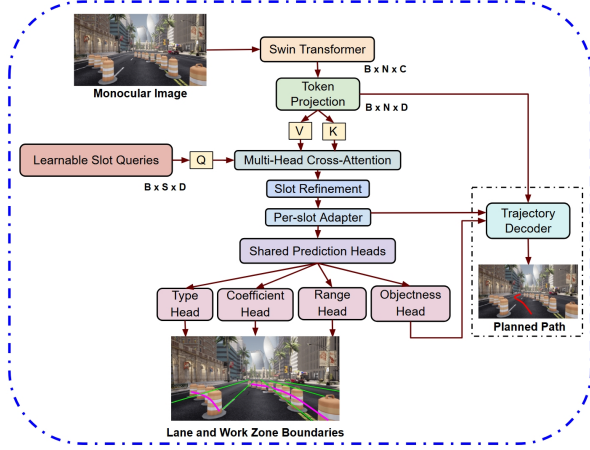


Figure 5. Overview of the *BoundaryFormer* architecture. In the *BF-IntegratedPlanner*, the driving trajectory is one of the outputs of the shared prediction heads. In the *BF-DistinctPlanner*, the shared prediction heads predict only the lane and work zone boundaries, with trajectory options predicted by the trajectory decoder. (B = Batch size, N = No. of spatial tokens, C = Backbone channel dimension, D = Hidden dimension of token projection block and learnable slot queries block, S = No. of slots, Q = Queries, K = Keys and V = Values)

5. The *BoundaryFormer* Architecture

We present a new transformer-based architecture called *BoundaryFormer* that predicts lane boundaries, work zone boundaries, and driving trajectories from a single RGB image using slot attention for structured scene parsing. An overview is given in Fig. 5. To keep the nomenclature fixed throughout the paper, *BF-IntegratedPlanner* denotes the integrated shared-slot design, in which trajectories are predicted by the shared slot-attention heads and *BF-DistinctPlanner* denotes the design with a distinct dedicated trajectory decoder (slots predicting only boundaries).

5.1. Backbone and Token Projection

An RGB image is processed by a pre-trained Swin Transformer backbone to extract spatial tokens. A two-layer token encoder projects these tokens into a shared latent space, stabilizing feature variance and improving interactions between visual tokens and learnable slot queries.

5.2. Slot Attention for Structured Entities

Learnable slot queries access the projected tokens via multi-head cross-attention to produce a set of slot features intended to correspond to lane boundaries, work zone boundaries, and (in *BF-IntegratedPlanner*) driving trajectories. Slot representations are then refined and passed through lightweight per-slot adapters. Shared prediction heads output object type, third-order polynomial coefficients, spatial range, and objectness scores for each predicted entity.

5.3. Slot-Based Boundary Prediction Heads

Each slot is decoded by four prediction heads. The coefficient head predicts cubic curve parameters for the boundary shape and position. The range head predicts the valid vertical start and end positions of the curve. The type head predicts the boundary class, such as lane boundary or work-zone boundary. The objectness head predicts whether the slot corresponds to a real boundary.

During training, Hungarian matching assigns predicted slots to ground-truth boundaries. Matched pairs supervise the coefficient, range, and type heads, while objectness is trained to distinguish matched boundary slots from unmatched slots.

5.4. Dedicated Trajectory Decoder in *BF-DistinctPlanner*

Modeling trajectories solely as one of the slots proves to be insufficient for long-horizon reasoning required for driving trajectory because slot attention tends to emphasize local spatial patterns (see Fig. 6). We therefore introduce a dedicated trajectory decoder operating in parallel with slot-based heads. The decoder aggregates global context via mean pooling of projected tokens and extracts boundary-related cues via objectness-weighted pooling of slot features. These representations are fused using a lightweight MLP to output K trajectory hypotheses (each with polynomial coefficients and spatial range) along with mode probabilities π_k for hypotheses $k = 1, \dots, K$; see Supplementary Sec. S1.2. The K alternatives are generated before route handoff; trajectory-only matching scores coverage, and π_k supports selection. This decouples object-level perception from global trajectory reasoning while retaining interpretable slot representations.

5.5. Training Objective

All *BF* outputs are jointly optimized using a composite matching-based loss over geometry, range, semantic type, objectness, false positives/negatives, and multimodal trajectory error. Hungarian assignment aligns the variable-size prediction set with ground truth. For details Refer Suppl.Sec. S1.

6. From *BF* to *BF++*: Work-Zone-Specific Robust Prediction

Our experiments in section 7 shows that *BF-DistinctPlanner*'s dedicated decoder is stronger than treating trajectories as interchangeable shared slots. *BF++* builds on this result by assigning boundary and trajectory reasoning explicit metric and semantic roles.

6.1. Topology-Oriented Design

Temporary devices cause drivable corridors to split, merge, shift, appear, and disappear. *BF++* represents this topology using calibrated ground-plane coordinates, non-interchangeable typed queries, 38 longitudinal anchors over

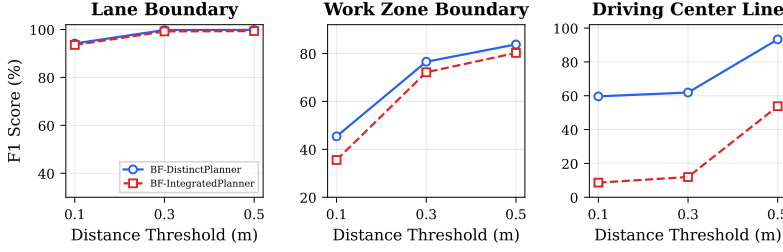


Figure 6. Per-class F1 in Experiment 1 across matching thresholds. BF-DistinctPlanner is the dedicated-decoder variant (blue circles), whereas BF-IntegratedPlanner predicts trajectories from shared slots (red squares).

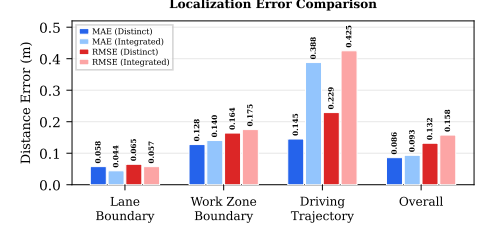


Figure 7. Matched-curve localization errors at the 0.5 m threshold in Experiment 1. Distinct denotes the dedicated decoder and Integrated denotes shared-slot trajectory prediction.

1.5–100 m with visibility and range, and two-stage curve reprojection into multi-scale image evidence. These components provide task-specific geometric inductive biases.

6.2. Camera and Camera+LiDAR Variants

BF++-Camera combines a ConvNeXt-Tiny trunk with a three-layer typed-query decoder (10 lane, 6 work-zone, and 1 trajectory query). The trajectory query emits two independently parameterized, confidence-scored centerlines. BF++-Camera+LiDAR adds only a 1.24M-parameter encoder for projected depth, height, and hit-mask features through a zero-initialized gate. The matched models contain 33.5M and 34.77M parameters. Full architectural, training, and multimodal-scoring details are in Supplementary Secs. S3.2–S3.4 and S3.6.

6.3. Closed-Loop Driving and Speed Control

During CARLA rollout, the route command selects the highest-confidence compatible BF++ centerline. A shared pure-pursuit controller tracks it with a 6 m lookahead and 2.85 m wheelbase. Boundary-midpoint lane snapping is disabled within 25 m of a detected work zone. A separate 32-channel utility LiDAR, rather than the learned 64-channel LiDAR input, controls speed for both variants. Its fifth-percentile corridor range drives a stopping-distance governor with an 8.9 m/s nominal speed, 2.0 m/s² deceleration, and 3.0 m buffer. A narrow-corridor range below 2.4 m commands a full stop. This shared non-PID controller isolates the learned-model comparison. Exact filters, equations, and actuation thresholds are in Supplementary Sec. S3.5.

7. Experiments and Results

7.1. Methodology and Evaluation Metrics

We perform experiments in four stages: (i) the original BF architecture comparison on a CARLA *TOWN01* 81K synthetic subset, (ii) scaling BF-DistinctPlanner to the original 137K+ Town01/Town10 training subset, (iii) BF synthetic-to-real fine-tuning, and (iv) BF++ camera and camera+LiDAR open-/closed-loop evaluation in

CARLA and real-world in-/out-of-distribution settings. The complete 149,478-frame synthetic release adds only the held-out Town02–05 data to the original 137,749-frame Town01/Town10 corpus described in Sec. 4. Open-loop curve-matching, localization, and trajectory-option metrics are defined in Supplementary Secs. S2.1 and S3.6. Closed-loop metric definitions and aggregation are provided in Supplementary Sec. S5.1.

7.2. Experiment 1: *TOWN01* 81K synthetic data architecture comparison

Following the definitions in Sec. 5, we compare **BF-IntegratedPlanner** (shared-slot trajectory prediction) against **BF-DistinctPlanner** (dedicated trajectory decoder). F1 and localization results are shown in Figs. 6 and 7, respectively. With a random 90/10 training/validation split, both variants nearly saturate lane-boundary localization at moderate thresholds. The driving-centerline results show a much larger gap: **BF-DistinctPlanner** substantially outperforms **BF-IntegratedPlanner** at strict distance thresholds. Work-zone boundary performance also favors **BF-DistinctPlanner**, though by a smaller margin. These results support separating global trajectory reasoning from the local slot outputs and directly motivate the explicit typed trajectory branch in BF++ (Sec. 6).

7.3. Experiment 2: **BF-DistinctPlanner** on Training and Evaluation on 137K+ Synthetic Data

We scale BF-DistinctPlanner to the original 137K+ synthetic training subset from CARLA *TOWN01* and *TOWN10*, retaining the random 90/10 split. At 0.5 m, lane boundaries achieve P/R/F1 = 82.2/88.6/85.3 and trajectories 76.7/75.2/75.9; work-zone boundaries remain harder at 58.8/53.1/55.8. See Supplementary Table S1 (Exp. 2) for threshold and localization results and Fig. S1 (left) for the 97.38%-accurate confusion matrix. Its dominant work-zone-to-lane confusion motivates BF++’s non-interchangeable typed queries.

Table 2. BF++ open-loop losses. ID = In Distribution Town01/Town10 validation split and OOD= Out-of-Distribution-Data from Town02,03,04,05.

Split	Model	Total	Obj.	Point	Route-ADE (m)
ID	BF++-Camera	2.298	0.110	2.097	0.081
ID	BF++-Camera+LiDAR	2.247	0.117	2.038	0.081
OOD	BF++-Camera	16.443	3.125	12.532	0.605
OOD	BF++-Camera+LiDAR	16.443	3.082	12.569	0.610

Table 3. BF++ matched-instance F1 at 0.5 m.

Split	Model	Lane F1	WZ F1	Traj. F1	Overall F1
ID	BF++-Camera	0.972	0.715	0.967	0.926
ID	BF++-Camera+LiDAR	0.971	0.721	0.967	0.927
OOD	BF++-Camera	0.515	0.174	0.400 [†]	0.447
OOD	BF++-Camera+LiDAR	0.495	0.164	0.402[†]	0.433

7.4. Experiment 3: Synthetic-to-Real Fine-Tuning

We next fine-tune the best Experiment 2 model on all 5,178 real-world samples with the same random 90/10 split. At 0.5 m, overall P/R/F1 is 60.7/91.9/73.1. Trajectories achieve 62.1/90.5/73.7 with 0.042/0.057 m MAE/RMSE, lanes 89.8/96.2/92.9, and work-zone boundaries 29.8/81.9/43.7. The easier trajectory geometry relative to the diverse synthetic scenarios reinforces the need for broad synthetic coverage. See Supplementary Table S1 (Exp. 3) for threshold and localization results and Fig. S1 (right) for 92.64% type accuracy, dominant work-zone-to-lane errors, and near-zero trajectory confusion.

7.5. Experiment 4: BF++ Camera and Camera+LiDAR

7.5.1. In-Distribution Open-Loop Results

Tables 2 and 3 report the ID comparison. Similar F1 and route error indicate saturation on familiar towns (Supplementary Sec. S3.7).

7.5.2. Out-of-Distribution Open-Loop Results

Tables 2 and 3 report the OOD comparison. Both variants degrade similarly, and LiDAR does not improve open-loop generalization in synthetic dataset (Supplementary Sec. S3.8).

7.5.3. Real-World Fine-Tuning Results

Table 4 reports the held-out real-world comparison. LiDAR modestly improves overall and work-zone F1, camera-only leads on lanes, and both tie on trajectories. Hence LiDAR data helps more in real-world than in simulation. (Supplementary Sec. S4).

7.5.4. Closed-Loop and Efficiency Results

Table 5 compares BF++ with SimLingo [12] and TransFuser++ (TF++) [7]. The benchmark now contains 76 scenarios and 228 weather-route evaluations. For a matched

Table 4. BF++ real-world fine-tuning with one scenario held out for testing and the other three scenarios used for fine-tuning.

Model	Overall F1@0.5	Lane F1@0.5	Work-zone F1@0.5	Trajectory F1@0.5
BF++-Camera	0.840	0.895	0.509	0.990
BF++-Camera+LiDAR	0.844	0.882	0.549	0.990

Table 5. Closed-loop and efficiency results. C/F/S are clear, fog, and storm Driving Scores over the 211-route common comparison. Efficiency is measured end to end on one NVIDIA RTX 5070 Ti.

(a) Closed-loop performance

Method	C	F	S	DS [†]	RC [†]	SR [†]	Coll.↓
BF++-Camera+LiDAR	68.0	63.7	61.6	64.4	87.0	34.3	47
BF++-Camera	65.8	64.1	59.0	63.0	87.3	31.3	45
SimLingo	60.1	59.8	57.9	59.3	74.0	37.4	71
TransFuser++	25.2	26.5	26.4	26.1	50.2	1.5	~244

(b) Efficiency

Model	Parameters	Latency	FPS
BF++-Camera	33.5M	9.6 ms	104.5
BF++-Camera+LiDAR	34.77M	11.3 ms	88.2
SimLingo	1,364.8M	727 ms	1.4
TransFuser++ (3-model ens.)	362M	50.9 ms	19.6

comparison, Table 5 retains the original 180-route campaign and the 31 Town10 routes common to all four methods at the evaluation freeze, giving 211 routes. BF++ outperforms both baselines at real-time rates. Full Town10 results and the complete 228-route audit are reported in Supplementary Sec. S5.2, Table S3, and Table S-A2.

8. Conclusion

This work introduced *WorkZonePlan*, a multimodal benchmark comprising 149,478 synthetic and 5,178 real-world samples with 3D lane, work-zone-boundary, and trajectory annotations, together with 76 CARLA work-zone scenarios and 228 weather-route evaluations for closed-loop testing. The accompanying *WAVE* pipeline supports scalable generation and annotation of these data. We also proposed a novel network called *BoundaryFormer* (BF). The BF ablation shows that long-horizon trajectory reasoning benefits from a dedicated decoder rather than treating trajectories as interchangeable scene slots. Building on this result, **BF++** incorporates metric ground-plane representations, typed boundary and trajectory queries, long-range anchors, curve refinement, and optional gated LiDAR fusion. Across synthetic in- and out-of-distribution tests, held-out real-world evaluation, and closed-loop driving, BF++ provides structured perception and planning at 88.2–104.5 FPS with only 33.5–34.77M parameters. On the 211 routes common to all methods at the evaluation freeze, its camera and camera+LiDAR variants achieve closed-loop Driving Scores of 63.0 and 64.4, respectively, exceeding SimLingo and TransFuser++; the complete 228-route results in Supplementary Sec. S5.2 preserve this ranking. Although broader out-of-distribution

generalization remains an important direction for further study, these results demonstrate that jointly learning work-zone geometry and route-compatible trajectory options is a practical and promising research direction for end-to-end planning in work zones.

References

- [1] Federal Highway Administration. Work zone management program. *U.S. Department of Transportation*, 2023. 2
- [2] Daniel Airinei, Elena Burceanu, and Marius Leordeanu. Inside knowledge: Graph-based path generation with explainable data augmentation and curriculum learning for visual indoor navigation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*, pages 1745–1754, 2025. 2
- [3] Jiping et. al. Bi. Lane detection for autonomous driving: Comprehensive reviews, current challenges, and future predictions. *IEEE Transactions on Intelligent Transportation Systems*, 2025. 4
- [4] Holger Caesar and et. al. nuscenes: A multimodal dataset for autonomous driving. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11621–11631, 2020. 3
- [5] Anurag et. al. Ghosh. Roadwork: A dataset and benchmark for learning to recognize, observe, analyze and drive through work zones. In *IEEE/CVF International Conference on Computer Vision*, 2025. 3, 4
- [6] Yanjun Huang and et. al. A survey on trajectory-prediction methods for autonomous driving. *IEEE transactions on intelligent vehicles*, 7(3):652–674, 2022. 4
- [7] Bernhard Jaeger, Kashyap Chitta, and Andreas Geiger. Hidden biases of end-to-end driving models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 8240–8249, 2023. 8
- [8] Xiaosong Jia, Zhenjie Yang, Qifeng Li, Zhiyuan Zhang, and Junchi Yan. Bench2drive: Towards multi-ability benchmarking of closed-loop end-to-end autonomous driving. In *Advances in Neural Information Processing Systems*, 2024. 4
- [9] Marius Leordeanu and Iulia Paraicu. Driven by vision: Learning navigation by visual localization and trajectory prediction. *Sensors*, 21(3):852, 2021. 2
- [10] Yueru Luo and et. al. Latr: 3d lane detection from monocular images with transformer. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7941–7952, 2023. 4
- [11] Maximilian Pittner and et. al. Sparselanestp: Leveraging spatio-temporal priors with sparse transformers for 3d lane detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025. 4
- [12] Katrin Renz, Long Chen, Elahe Arani, and Oleg Sinavski. Simlingo: Vision-only closed-loop autonomous driving with language-action alignment. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11993–12003, 2025. 8
- [13] Sahu and et. al. "SafeRoute: A Risk-Minimizing Real-Time Route Planner for Autonomous Vehicles". In *IEEE ITSC*, 2024. 2
- [14] Nishad Sahu and et. al. Towards the safe operation of autonomous vehicles in work zones. In *2025 IEEE Intelligent Vehicles Symposium (IV)*, pages 2380–2387. IEEE, 2025. 2
- [15] Weijing Shi and Ragunathan Raj Rajkumar. Work zone detection for autonomous vehicles. In *2021 IEEE International Intelligent Transportation Systems Conference (ITSC)*, pages 1585–1591. IEEE, 2021. 2, 3, 4
- [16] S Sural, G T Su, N Sahu, Naren, and R Rajkumar. Cosim: A co-simulation framework for testing avs in adverse operating conditions. In *26th IEEE ITSC 2023*, 2023. 2, 4
- [17] Shounak Sural, Nishad Sahu, and Ragunathan Rajkumar. Workzone3d: A multimodal dataset for 3d work zone perception in autonomous driving. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 3972–3981, 2026. 3, 5
- [18] Zhiran Yan and et. al. Framework and multi-modal dataset for roadwork zone detection and geo-localization. In *2025 IEEE Intelligent Vehicles Symposium (IV)*, pages 1758–1765. IEEE, 2025. 3
- [19] Fisher Yu and et. al. Bdd100k: A diverse driving dataset for heterogeneous multitask learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2636–2645, 2020. 3

Supplementary Material

S1. Original BF Objective and Implementation

This supplement preserves the complete specification and results of the original **BoundaryFormer (BF)** while separating them from its successor, **BoundaryFormer++ (BF++)**. BF has two variants. In every section and figure, **BF-IntegratedPlanner** denotes trajectory prediction from the shared slot-attention heads, and **BF-DistinctPlanner** denotes the dedicated trajectory decoder, with shared slots reserved for boundaries.

S1.1. Original BF architecture

The original BF processes one RGB image with a pretrained Swin Transformer and projects its spatial tokens into a shared latent space. Learnable slots attend to those tokens and produce cubic coefficients, valid longitudinal range, semantic type, and objectness for lane and work-zone boundaries. In BF-IntegratedPlanner, those same slot heads also predict trajectory options. In BF-DistinctPlanner, a separate global decoder fuses mean-pooled visual context with objectness-weighted boundary-slot context and emits K trajectory hypotheses and mode probabilities. The latter design preserves object-level boundary representations while giving long-horizon trajectory reasoning a dedicated pathway.

S1.2. Multimodal trajectory options in BF

The trajectory output is a candidate set, not a single curve duplicated across slots. The annotation schema retains every locally valid trajectory option, including both alternatives when a scene admits two. Let $\hat{\mathcal{T}} = \{(\hat{T}_k, \pi_k)\}_{k=1}^K$ denote the predicted curves and normalized mode probabilities. BF-IntegratedPlanner forms this set from slots classified as trajectories; BF-DistinctPlanner produces it directly with the dedicated decoder. In both variants, all options are generated from perception before any route-level command is applied. A route planner may then use command compatibility and π_k to select one option, but it neither changes the predicted geometry nor reveals the ground-truth best mode.

S1.3. Complete BF matching objective

All BF outputs are jointly optimized with Hungarian assignment. Matching uses semantic compatibility and geometric similarity between cubic coefficients and valid spatial ranges. For each matched prediction-ground-truth pair, weighted Smooth- L_1 losses supervise coefficients and range, cross-entropy supervises type, and sampled points on the curves provide an alignment loss. Unmatched predictions and ground-truth instances incur false-positive and false-negative penalties. Objectness uses binary cross-entropy with logits. The full objective is

$$\mathcal{L}_{\text{BF}} = \mathcal{L}_{\text{coeff}} + \mathcal{L}_{\text{range}} + \mathcal{L}_{\text{type}} + \mathcal{L}_{\text{curve}} + \mathcal{L}_{\text{obj}} + \mathcal{L}_{\text{FP}} + \mathcal{L}_{\text{FN}} + \lambda_{\text{traj}} \mathcal{L}_{\text{traj}}. \quad (\text{S1})$$

Here, $\mathcal{L}_{\text{traj}}$ is active for BF-DistinctPlanner and supervises the dedicated multimodal decoder. Trajectory assignment is restricted to predicted and annotated trajectory sets, so a trajectory option never competes with a lane or work-zone boundary in the matching. Matched options supervise curve geometry and their

BF-DistinctPlanner — Type Classification Confusion Matrices

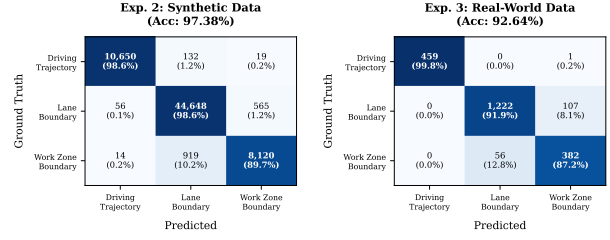


Figure S1. Type-classification confusion matrices for BF-DistinctPlanner in Experiments 2 and 3 at the 0.5 m distance threshold.

mode scores; unsupported options are penalized through confidence/objectness supervision. This is the original BF objective; it is not reused to rename or retroactively redefine BF++.

S2. Complete Original BF Results

Table S1 contains the threshold-wise BF-DistinctPlanner results. Experiment 2 uses the original 137K Town01/Town10 training subset with the paper’s 90/10 split; Experiment 3 fine-tunes on all 5K real-world samples with the same 90/10 split.

S2.1. Open-loop evaluation metrics

We apply standard detection statistics to distance-thresholded, instance-level matching for lane boundaries, work-zone boundaries, and the BF trajectory-option set. Each predicted and annotated cubic curve $x = ay^3 + by^2 + cy + d$ is sampled at 50 points over its overlapping longitudinal range. The mean 3D Euclidean point distance is the pairwise cost. Evaluation greedily forms one-to-one matches in ascending cost order at $\tau \in \{0.1, 0.3, 0.5\}$ m. A matched pair is a true positive (TP), an unmatched prediction is a false positive (FP), and an unmatched annotation is a false negative (FN). We compute

$$P = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad R = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad F1 = \frac{2PR}{P + R}, \quad (\text{S2})$$

with F1@0.5 m primary. Mean absolute error (MAE) and root mean squared error (RMSE) summarize the matched 3D curve distances at 0.5 m, while the per-type confusion matrix measures lane/work-zone/trajectory identity errors. For a trajectory row, P is option precision and R is option coverage; the multimodal and mode-selection metrics are defined in Sec. S3.6. BF++ uses 38 metric anchors rather than 50 polynomial samples. Its validation summary also reports the total objective from Eq. S4, its objectness and point-regression terms, and route-selected ADE over the 38 anchors. The corresponding partition-specific results are reported in Tables 2–4.

S3. BF++ Architecture and Training

S3.1. Design lineage

BF++ follows the conclusion of the original BF ablation: trajectories require an explicit global role rather than an interchangeable

Table S1. Detection and localization performance of BF-DistinctPlanner across the original experiments. Precision (P), Recall (R), and F1 are percentages at $\tau \in \{0.1, 0.3, 0.5\}$ m. MAE and RMSE are in meters and use matched pairs at $\tau = 0.5$ m.

Experiment	Boundary Type	$\tau = 0.1$ m			$\tau = 0.3$ m			$\tau = 0.5$ m			Loc. Error @ 0.5 m	
		P	R	F1	P	R	F1	P	R	F1	MAE	RMSE
Exp. 2	Lane Boundary	9.8	10.6	10.2	65.1	70.1	67.5	82.2	88.6	85.3	0.2258	0.2486
	Work Zone Boundary	9.9	8.9	9.4	41.6	37.5	39.5	58.8	53.1	55.8	0.2290	0.2612
	Driving Trajectory	47.5	46.6	47.0	68.3	66.9	67.6	76.7	75.2	75.9	0.1291	0.1711
	Overall	16.1	16.5	16.3	61.5	63.0	62.2	77.2	79.1	78.2	0.2095	0.2388
Exp. 3	Lane Boundary	26.5	28.4	27.4	76.0	81.4	78.6	89.8	96.2	92.9	0.1763	0.2091
	Work Zone Boundary	4.6	12.6	6.7	22.2	60.9	32.5	29.8	81.9	43.7	0.2167	0.2467
	Driving Trajectory	59.0	86.0	70.0	61.8	90.1	73.4	62.1	90.5	73.7	0.0419	0.0565
	Overall	24.4	37.0	29.4	52.0	78.8	62.6	60.7	91.9	73.1	0.1483	0.1864

Table S2. From original BF to BF++.

Property	BF	BF++
Coordinates	image-token latent space	calibrated metric ground plane
Entity roles	shared set slots	typed lane/WZ/trajectory queries
Geometry	cubic coefficients/range	38 forward anchors, visibility/range
Trajectory	distinct global decoder	explicit typed query, two options
Evidence	one-pass slot decoding	two-stage curve reprojection/refinement
Modalities	camera	camera or gated camera+LiDAR

shared slot. Table S2 summarizes the resulting redesign. The contribution is a work-zone-specific set of geometric and topological inductive biases, not the individual use of an existing backbone, attention layer, or matching routine.

The physical representation follows the structure of a reconfigured road. Typed lane and work-zone queries preserve the two sides of the temporary drivable corridor; visibility and valid range allow curves to start and stop; metric anchors preserve forward continuity as corridors shift, split, or merge; and the typed trajectory output remains globally distinct from boundary instances. These constraints prevent permutation-based matching from exchanging physical roles that are not interchangeable in a work zone.

S3.2. BF++-Camera

RGB is resized from 1920×1080 to 768×432 and normalized. An ImageNet-pretrained ConvNeXt-Tiny produces stride-4/8/16/32 features. Stride-16 and stride-32 maps are projected by 1×1 convolution and GroupNorm to $d = 256$, then summed at 48×27 . Camera intrinsics map every token to a flat-ground forward/lateral coordinate used as metric positional encoding. The CARLA camera has a 90° field of view and is mounted at (1.6, 0, 1.7) m with -15° pitch.

A three-layer Transformer decoder ($d = 256$, dropout 0.1) uses 17 typed queries. These comprise one trajectory query at zero lateral offset, four anchored lane queries at $-6, -2, +2, +6$ m with a 2.5 m residual scale, six free lane queries at $-9, -4.5, 0, +4.5, +9, 0$ m with a 9 m residual scale, and six work-zone queries at $\pm 1, \pm 3, \pm 5$ m. The repeated zero-offset free-lane query is intentional and provides a second wide-

residual center slot. The dedicated trajectory query simultaneously emits two route-selectable centerline options. Each option has independently regressed geometry and a confidence score; the two curves share global query context but neither is generated conditionally from the other. Each curve uses 38 forward anchors spanning 1.5–100 m. Heads predict lateral offsets, per-anchor visibility, objectness, and a valid forward range. Two refinement stages sample stride-4/8/16 image evidence along the projected curve at heights $\{0, 0.5, 1.0\}$ m and regress lateral and visibility corrections.

S3.3. BF++-Camera+LiDAR

The perception LiDAR has 64 channels, vertical field of view $+10^\circ / -30^\circ$, 100 m range, approximately 16K points/frame at 20 Hz, and is co-located with the camera. A synchronized sweep is projected through fixed LiDAR-to-camera extrinsics and z-buffered at 768×432 into three channels: depth clip($d, 0, 60$)/60, height clip($(h + 1)/4, 0, 1$), and a hit mask. Four stride-2 3×3 convolution–GroupNorm–GELU blocks of width 48/96/192/256 and a 3×3 head produce a 48×27 embedding.

Fusion is deliberately conservative:

$$\mathbf{f} \leftarrow \mathbf{f} + \mathbf{g} \odot E_L(\mathbf{r}), \quad \mathbf{g} \in \mathbb{R}^{256}, \quad \mathbf{g}_0 = \mathbf{0}. \quad (\text{S3})$$

The zero gate makes the initial fused model identical to its camera parent, and the learned correction is admitted channel by channel. Decoder, heads, and refiner are otherwise unchanged. BF++-Camera has 33.5M parameters; BF++-Camera+LiDAR has 34.77M, of which 1.24M belong to the LiDAR branch.

S3.4. Loss and optimization

Ground-truth polynomials are sampled at the 38 metric anchors. Predictions are matched independently within each entity type using masked mean point distance, so lane, work-zone, and trajectory identities cannot exchange through matching.

The BF++ objective is

$$\mathcal{L}_{\text{BF++}} = 5\mathcal{L}_{\text{point}} + \mathcal{L}_{\text{vis}} + 2\mathcal{L}_{\text{obj}} + 0.5\mathcal{L}_{\text{range}} + \mathcal{L}_{\text{ref}}, \quad (\text{S4})$$

where $\mathcal{L}_{\text{ref}}$ supervises the two-stage curve corrections. The point term receives a further $2.5 \times$ weight on work-zone curves. The “Point” and “Obj.” entries in Table 2 are already weighted contributions, rather than raw losses. Together with the visibility, range,

and refinement contributions, they sum to the reported total. For example, the OOD camera row gives $12.532 + 3.125 + 0.423 + 0.009 + 0.355 = 16.443$. The applied weights are 5.0 for point regression, with the additional work-zone factor above, 2.0 for objectness, 1.0 for visibility, and 0.5 for range. The fused model initializes all camera weights from BF++-Camera. For epochs 0–2, the camera trunk is frozen while the LiDAR branch and gate train at 2×10^{-4} . For epochs 3–11, decoder, heads, and refiner are unfrozen at one-quarter of that learning rate while the backbone remains frozen. We use AdamW, weight decay 0.05, batch size 12, gradient clipping at 1.0, no geometric augmentation, and Gaussian image noise $\sigma = 0.02$. The fixed training and validation partitions are summarized in Sec. S6.

S3.5. Common route command, controller, and speed governor

The typed trajectory query produces two options. At evaluation, every model receives the same benchmark expected-route command (left/right/straight). Both curves are decoded before hand-off. The command masks incompatible options, and the controller selects the highest-confidence remaining option. The command is therefore analogous to a destination-level instruction. It is not an encoder input, does not expose local boundary geometry, and cannot select the lowest-error option after ground truth or execution is observed.

The selected centerline is tracked by pure pursuit. For lateral aim-point offset y_{aim} , lookahead $\ell = 6$ m, and wheelbase $L = 2.85$ m, the steering angle and normalized command are

$$\begin{aligned} \delta &= \text{atan2}(2Ly_{\text{aim}}, \ell^2 + y_{\text{aim}}^2), \\ \tilde{u}_s &= \text{clip}\left(\frac{\delta}{70^\circ}, -0.7, 0.7\right). \end{aligned} \quad (\text{S5})$$

The command is exponentially smoothed with coefficient 0.6. When the predicted left and right lane boundaries form a geometrically consistent adjacent pair, lane snapping places the reference centerline at their midpoint. Lane snapping is enabled in the deployed configuration but is suppressed within 25 m of a detected work zone, where the predicted work-zone trajectory is retained.

Both BF++ variants use the same separate 32-channel utility LiDAR for longitudinal control. This sensor is distinct from the 64-channel perception LiDAR available only to BF++-Camera+LiDAR. In the utility-LiDAR frame, x is forward, y is right, and z is up. The base filter retains points satisfying $-2.2 < z < 0.4$ m and $2 < x < 35$ m. The center corridor further requires $|y| \leq 1.4$ m. If it contains at least four points, the obstacle distance d is the fifth percentile of their forward ranges x . This statistic is less sensitive to isolated returns than the absolute minimum. A narrow emergency corridor uses $|y| \leq 0.9$ m and $x < 20$ m. Its distance d_{nar} is likewise the fifth percentile when at least three points are present.

The stopping-distance governor sets

$$v_{\text{lim}} = \sqrt{\max(0, 2a(d - d_{\text{stop}}))}, \quad v_{\text{des}} = \min(v_{\text{nom}}, v_{\text{lim}}), \quad (\text{S6})$$

with $a = 2.0$ m/s², $d_{\text{stop}} = 3.0$ m, and deployed nominal speed $v_{\text{nom}} = 8.9$ m/s. All reported runs set BF_TARGET_SPEED=8.9; the 11.2 m/s code fallback was not used. The narrow-corridor condition $d_{\text{nar}} < 2.4$ m overrides Eq. S6 and sets $v_{\text{des}} = 0$. Time to

collision is logged for analysis but does not enter the active speed limiter.

Let $e = v_{\text{des}} - v$ denote speed error. Throttle u_t and brake u_b are generated by the custom rule

$$\begin{aligned} u_t &= \begin{cases} \text{clip}(0.55e, 0, 0.75), & e \geq 0, \\ 0, & e < 0, \end{cases} \\ u_b &= \begin{cases} 1, & v_{\text{des}} < 0.15 \wedge v > 0.3, \\ \text{clip}(-0.6e, 0, 1), & e < -0.4, \\ 0, & \text{otherwise.} \end{cases} \end{aligned} \quad (\text{S7})$$

Thus, negative errors within 0.4 m/s use a coast dead band. This controller is neither a PID nor CARLA’s built-in vehicle controller. BF++-Camera describes the learned model’s input modality; its complete evaluated driving stack additionally uses the non-learned utility-LiDAR speed governor. Holding that governor fixed makes the two BF++ variants directly comparable. It does not, by itself, isolate the contribution of the learned architecture relative to SimLingo or TransFuser++, whose native speed-control systems differ.

S3.6. Trajectory-option metrics

Let $\mathcal{T}^* = \{T_j^*\}_{j=1}^M$ be the annotated valid options and $\hat{\mathcal{T}} = \{(\hat{T}_k, \pi_k)\}_{k=1}^K$ the predictions. For the common visible anchor set $\mathcal{A}$, their mean point distance is

$$d(\hat{T}_k, T_j^*) = \frac{1}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \|\hat{T}_k(a) - T_j^*(a)\|_2. \quad (\text{S8})$$

If $\mathcal{M}_\tau$ is the one-to-one trajectory matching at threshold τ , option coverage and option precision are

$$\text{Cov@}\tau = \frac{|\mathcal{M}_\tau|}{M}, \quad \text{Prec}_{\text{opt}}@ \tau = \frac{|\mathcal{M}_\tau|}{K}. \quad (\text{S9})$$

Accordingly, recall and precision in the trajectory rows of Table S1 are exactly option coverage and option precision, respectively; matched MAE/RMSE measure the geometry of recovered options. We also define the oracle hypothesis diagnostic

$$\text{minADE} = \frac{1}{M} \sum_{j=1}^M \min_k d(\hat{T}_k, T_j^*). \quad (\text{S10})$$

This metric measures whether the candidate set contains a good option, but it does not measure whether the model or route planner selects it. With $k_{\text{top1}} = \arg \max_k \pi_k$ and $k_{\text{route}} = \arg \max_{k \in \mathcal{C}(c)} \pi_k$, where $\mathcal{C}(c)$ is the subset compatible with route command c , top-1 ADE and route-selected ADE compare those respective curves with the command-compatible annotation. BF++ reports route-selected ADE and executes that same mode in closed loop; it never uses minADE or an oracle best-of-two mode for the deployment claim.

We additionally evaluated the frozen trajectory head on the fixed 2,791-frame validation split. Among frames with valid trajectory diagnostics, the 2,745 single-option cases give a top-1 ADE of 0.081 m (median 0.025 m) and an oracle minADE of 0.075 m. For the 35 multi-option cases, top-1 ADE is 0.246 m, oracle minADE is 0.124 m, and both annotated options are recovered within 0.5 m in 30/35 cases. Mode-count accuracy is 98.7%

for single-option frames and 100% for multi-option frames. These oracle values diagnose the coverage of the predicted set; they are not used for route handoff or closed-loop scoring.

S3.7. In-distribution open-loop analysis

The two final BF++ variants use the same fixed 2,791-frame Town01/Town10 validation split. Tables 2 and 3 provide the split-by-sensor comparison. Route-ADE is the mean error of the command-selected trajectory over 38 anchors and is not an oracle best-of-two result. The fused model slightly reduces total and point losses, while both variants obtain the same Route-ADE and nearly identical overall F1. These results indicate that open-loop performance is saturated on familiar towns.

S3.8. Additional open-loop evaluation

The OOD evaluation separates missed detections from localization error. Lane F1 decreases from 0.97 in distribution to 0.50 OOD, while OOD precision remains 0.72 and recall decreases to 0.52. Among matched curves, mean point error increases more moderately, from 0.08 m to 0.20 m. Work-zone detection is also sensitive to the confidence threshold learned in simulation: on Town02, lowering the threshold from 0.30 to 0.03 raises work-zone F1 from 0.08 to 0.36 without changing the predicted curves. Town02 remains the hardest split because its complex turns and geometry. LiDAR is approximately neutral in the aggregate OOD open-loop results. This pattern is consistent with the closed-loop failure analysis, in which 77% of collisions occur in Towns02 and 05.

S4. BF++ Real-World Fine-Tuning Details

S4.1. Physical sensors and fused model input

The real subset is drawn from WorkZone3D. Its physical rig uses a telephoto forward camera and six 16-channel VLP-16 LiDAR units. The released KITTI-format data fuse the six units into one ego-registered point cloud per frame. BF++ therefore consumes the merged cloud, rather than a single physical 64-channel sensor. A representative release frame contains approximately 43K points, spans -6.1° to $+18.0^\circ$ in elevation, and occupies approximately 206 distinct 0.1° elevation bins. This density and angular support are consistent with the six differently mounted VLP-16 sensors.

The native forward RGB stream is 1024×768 with a $25.3^\circ \times 17.3^\circ$ field of view. Depending on the capture sequence, camera/LiDAR acquisition is 30/20 Hz or 10/10 Hz. Precise timestamps and the 6-DoF UTM vehicle state support camera-LiDAR synchronization and alignment.

We content-matched the experiment images to the release’s `image_1` camera and selected its calibration by direct LiDAR-to-image reprojection. At the 768×432 model input, the recovered intrinsics are $f_x = 1747.3$, $f_y = 1318.1$, $c_x = 387.1$, and $c_y = 235.3$. The extrinsics place the camera 2.04 m forward and 1.14 m above ground with -1.62° pitch. Projection through the release calibration places 653 points in the image on the expected lane markings, work-zone objects, and vehicles, compared with 4 and 9 in-image points for the two alternative calibrations. The same calibration is used during fine-tuning and evaluation.

The merged point cloud is projected using `P2` and `Tr_velo_to_cam`, then rasterized at 768×432 . Its three input channels are depth normalized by 60 m, height above ground over

Table S-A1. Read-only consistency audit of the real-world annotations. Counts are entity-level; “correct” is the fraction requiring no correction.

Class	Inspected	Missing	Spurious	Correction	Correct
Lane boundary	~470	0	0	0	100%
Work-zone boundary	~135	1	0	8	93.3%
Trajectory	~154	0	0	2	98.7%

$[-1, 3]$ m, and a z-buffered hit mask. These channel definitions are identical to those used for the simulated perception LiDAR.

S4.2. Leave-one-scenario-out protocol

The raster-paired subset contains 5,178 released frames from four independent capture sequences, which we denote Scenarios 1–4. The result reported in Table 4 holds out Scenario 2, a nighttime sequence containing 773 frames, for testing and uses Scenarios 1, 3, and 4 for fine-tuning. We fine-tune the complete network for eight epochs with the backbone learning rate set to 0.1 times the head learning rate, batch size 12, AdamW, and weight decay 0.05.

S4.3. Annotation consistency audit

We performed a read-only audit on 152 frames sampled at uniform temporal intervals, with 38 frames from each scenario. Released lane, work-zone, and trajectory labels were projected onto the corresponding RGB image using the verified calibration. Every visible labeled entity was classified as correct, requiring correction when the apparent offset exceeded approximately 0.3 m, or spurious; visible but unlabeled entities were counted as missing. The audit uses the exact labels from the reported experiments. No labels were changed and no model was retrained.

The audit was a single-rater, AI-assisted screening in which a vision-capable model inspected every projected overlay. Its judgments concern apparent image alignment and should not be interpreted as centimetre-level 3D measurements. Geometric support is claimed only where LiDAR supplies an independent reference, such as work-zone-object clusters and the reprojection check above.

Overall, 143/152 frames (94.1%) contain no audit flag. The nine flagged frames cluster in two cases: ambiguous taper geometry in Scenario 1, where a labeled closure boundary crosses lane dashes that appear open in the image, and far-range polynomial-tail curvature beyond 15 m in Scenario 4. Near-field alignment within 15 m is consistent across all sampled frames. In the fold where Scenario 4 is held out, its weaker performance appears primarily photometric, owing to darkness and glare, rather than the result of systematic annotation displacement. Two sampled frames contain valid dual-trajectory forks, and both alternatives are labeled.

S5. Closed-Loop Evaluation Details

S5.1. Protocol and metric aggregation

Closed-loop evaluation follows the CARLA/Bench2Drive protocol. The benchmark contains 60 work-zone scenarios across CARLA Towns01–05 and 16 additional in-distribution scenarios

in Town10. Replaying all 76 scenarios under clear-day, foggy-dusk, and storm-night conditions gives 228 scheduled routes per method. For route r ,

$$RC_r = 100 \frac{d_r^{\text{completed}}}{d_r^{\text{route}}}, \quad DS_r = RC_r IP_r, \quad (S11)$$

where $IP_r \in [0, 1]$ is the accumulated infraction penalty. Success Rate is the percentage of routes satisfying the benchmark’s full-route success condition, and the collision count is accumulated over the evaluated routes. We report weather-specific and pooled results together with paired route-level Driving Score comparisons. The route command selects a compatible trajectory before rollout, as detailed in Sec. S3.5; no metric selects an option after observing ground truth or execution outcomes.

An agent termination caused by blocking, route deviation, or a collision is treated as a completed evaluation and contributes its penalized Driving Score to every aggregate. A simulator or harness failure produces no valid endpoint score and is rerun under the same configuration and seed. No route is excluded because of driving performance. At the evaluation freeze, 211 routes had valid results for all four methods. After that freeze, the remaining simulator or harness failures were rerun. The completed audit therefore contains one valid endpoint for each of the $4 \times 228 = 912$ scheduled method–route records.

Table 5 pools the original 180-route campaign with the 31 Town10 routes common to all methods at the evaluation freeze. Table S-A2 instead includes all 48 completed Town10 routes, giving 228 records per method. BF++-Camera+LiDAR has the highest Driving Score in both comparisons, while SimLingo has the highest Success Rate.

S5.2. Town10 in-distribution extension

Town10 adds 16 work-zone scenarios and 48 scheduled weather routes per model. The interrupted evaluations were rerun under the same harness, configuration, and seeds, giving 48/48 valid records for every method. Table S3 reports all 16 routes in each weather.

BF++-Camera+LiDAR leads BF++-Camera by 3.6 Driving Score and records no collisions on Town10. SimLingo has the highest perfect-route count, but lower Driving Score and route completion than the fused BF++ model. Table 5 uses the 31-route Town10 intersection available at the evaluation freeze; Table S-A2 pools all 48 completed records with the original 180 routes.

S5.3. Runtime and paired-route analysis

Both BF++ variants were trained on one NVIDIA RTX 5090 and evaluated under the same deployment configuration on one NVIDIA RTX 5070 Ti. Runtime includes preprocessing and curve decoding. The LiDAR raster and encoder add 1.7 ms while retaining faster-than-real-time operation. Parameter counts are exact checkpoint sums. TransFuser++ uses a three-checkpoint ensemble. SimLingo uses camera/IMU/GNSS, whereas TransFuser++ uses camera/LiDAR/IMU/GNSS. The separate utility LiDAR is the common stopping governor for both BF++ variants and is not a learned BF++-Camera input.

In the original Towns01–05 campaign, BF++-Camera+LiDAR changes paired-route Driving Score by +1.09 relative to BF++-Camera, but the difference is not significant ($t = 1.09$). On the completed Town10 set, the difference is +3.6 DS. Across all

228 routes, BF++-Camera+LiDAR obtains 64.5 DS and BF++-Camera obtains 62.8. SimLingo and TransFuser++ obtain 60.5 and 26.0, respectively. The BF++ variants therefore record higher Driving Scores and fewer collisions than the external baselines. SimLingo, however, has the highest Success Rate, with 87/228 successes compared with 76/228 for BF++-Camera+LiDAR.

S5.4. External-baseline configuration and scope

The evaluated checkpoints, sensors, navigation adapters, controllers, and inference settings are summarized here. SimLingo uses the authors’ public epoch=013 checkpoint (SHA-256 prefix `ec894372`) with its native sensors and controller and is not fine-tuned on WorkZonePlan. TransFuser++ uses the authors’ public all-towns three-checkpoint ensemble (prefix `d6fbdc28`), with all members evaluated at each step. The BF++-Camera+LiDAR checkpoint has prefix `b47fd028`; the BF++-Camera control export has prefix `130eb363`. All methods receive the same benchmark route command and are evaluated on the same machine and harness with Epic rendering quality, a 600 s timeout, and one attempt per route. Because the external methods retain their native speed controllers, the fixed BF++ utility-LiDAR governor controls the camera-versus-fusion comparison but does not independently isolate the learned architecture from controller differences across model families.

S5.5. Scenario-family analysis and paired uncertainty

We retain the pre-rerun 208-route audit subset for the previously computed scenario-family analysis and group it using generator metadata when available, including `route_type` in Towns03–05 and the Town01 layout variants. For remaining routes, a disclosed geometric rule assigns shift-and-return paths with lateral displacement above 2.5 m to the detour-like family, paths with net lateral displacement above 2.5 m to lane-change/merge, and the remainder to straight. Table S-A3 reports Driving Score, integer successes, and collision events for each family. This diagnostic subset is not used for the 211-route comparison or the complete 228-route aggregate in Table S-A2.

The largest gain occurs in the merge–detour–return family, where BF++-Camera+LiDAR exceeds SimLingo by 13.4 Driving Score and records 29 rather than 47 collisions. SimLingo is stronger on the clean lane-change/merge and Town01 merge-left-return families, which also explains its higher aggregate Success Rate.

We also perform a layout-blocked bootstrap with 10,000 resamples over 64 layouts. All weather replays from a layout are resampled as one unit. BF++-Camera+LiDAR minus SimLingo has mean $\Delta DS = +5.04$, with 95% confidence interval $[-2.86, +12.98]$ and two-sided $p \simeq 0.20$. BF++-Camera minus SimLingo has mean $\Delta DS = +3.72$, interval $[-3.92, +11.86]$, and $p \simeq 0.35$. The paired differences are positive but are not statistically significant at the layout level with 64 effective units.

A qualitative inspection of an evaluated storm-night Town03 detour success found that BF++-Camera+LiDAR completed the route with 100.0 DS while SimLingo obtained 29.0 DS. In representative Town02 and Town04 failures, the agents left the route before cone contact, consistent with the route-level OOD failure pattern discussed in Sec. S3.8.

Table S-A2. Complete 228-route closed-loop results. Table 5 reports the 211 routes common to all four methods at the evaluation freeze, whereas this table uses every route after completing the interrupted evaluations. Success (“Perfect”) is defined as $DS \geq 99.9$; SR reports the exact count and percentage.

Metric	BF++-Camera+LiDAR	BF++-Camera	SimLingo	TransFuser++
Valid records	228/228	228/228	228/228	228/228
Clear DS	68.7	66.3	60.6	25.4
Fog DS	64.3	63.8	60.6	26.3
Storm DS	60.6	58.4	60.3	26.1
Overall DS	64.5	62.8	60.5	26.0
RC (%)	87.1	87.5	74.1	49.4
SR	76/228 (33.3%)	69/228 (30.3%)	87/228 (38.2%)	3/228 (1.3%)
Collisions	47	47	71	261

Table S-A3. Closed-loop scenario-family breakdown on the pre-rerun 208-route audit subset. Each entry is Driving Score / successes / collision events.

Family (<i>n</i> routes/model)	BF++-Camera+LiDAR	BF++-Camera	SimLingo	TransFuser++
Merge–detour–return (81)	63.7/32/29	63.3/31/31	50.3/22/47	28.1/0/104
Lane-change/merge (85)	59.2/17/8	58.1/15/4	60.5/34/11	24.6/0/82
Town01 straight open lane (15)	85.4/11/3	82.5/9/2	73.6/10/4	23.3/0/16
Town01 merge-left-return (12)	81.4/8/0	70.5/7/0	92.5/8/1	14.3/0/26
Shift-and-return/detour-like (9)	44.9/0/7	47.2/0/7	42.9/0/7	15.2/0/12
Straight within lane (6)	85.0/3/0	85.0/3/0	83.1/4/1	63.0/3/3

Table S3. Town10 closed-loop results on all 48 routes for each method. C/F/S are clear, fog, and storm Driving Scores.

Method	DS	RC	Coll.	Perfect	C/F/S
BF++-Camera+LiDAR	68.8	88.7	0	15/48	73.2/73.7/59.4
BF++-Camera	65.2	88.1	2	13/48	70.1/69.2/56.4
SimLingo	63.4	82.4	1	20/48	54.4/61.2/74.7
TransFuser++	23.1	48.0	49	0/48	22.2/23.9/23.2



Figure S2. The work-zone object classes in *WorkZonePlan*.

S6. Town-Wise Dataset and Closed-Loop Distribution

Table S4 separates released frames from closed-loop routes. Following the accounting used throughout the paper, the release contains 137,749 Town01/Town10 frames and 11,729 Town02–05 frames, giving 149,478 synthetic and 5,178 real samples (154,656 total). Towns01–05 contribute 12 closed-loop scenarios each, while Town10 contributes 16 additional in-distribution scenarios. Three weather replays per scenario produce 228 scheduled routes from 76 base layouts.

The nine work-zone object classes used to construct the synthetic scenarios are shown in Fig. S2.

Table S4. Frame and closed-loop distribution. Scen. counts base layouts; Routes counts their three weather replays.

Town/source	Frames	Role	Scen.	Routes
Town01	80K+	ID corpus	12	36
Town10	50K+	ID corpus	16	48
Reported ID subtotal	137,749	–	28	84
Town02	2,797	OOD holdout	12	36
Town03	4,945	OOD holdout	12	36
Town04	1,792	OOD holdout	12	36
Town05	2,195	OOD holdout	12	36
Synthetic total	149,478	–	76	228
WorkZone3D real	5,178	Real transfer	–	–
Complete dataset	154,656	–	76	228

The BF++ experiments use 125,626 training frames and a fixed 2,791-frame validation subset drawn from the reported 137,749-frame Town01/Town10 corpus. The remaining 9,332 ID frames are retained in the corpus but are not used in the fixed BF++ train/validation comparison. The 11,729 Town02–05 frames are reserved for held-out evaluation and are added only when forming the 149,478-frame synthetic release total. These counts define the fixed training, validation, and held-out partitions used for the reported BF++ experiments.